flutter_invoice_generator

Learning Guide - how the package is designed and built, phase by phase, in simple language

Chapter 1: Phase 1 - Architecture Plan
Chapter 2: Phase 2 - Package Skeleton and Models

Chapter 3: Phase 3 - Calculation Engine
Chapter 4: Phase 4 - PDF Generation
Chapter 5: Phase 5 - Templates and Customization
Chapter 6: Phase 6 - Preview Widget
Chapter 7: Phase 7 - Save, Share, Print
Chapter 8: Phase 8 - Example App
Chapter 9: Phase 9 - Documentation
Chapter 10: Final audit before release
Status: audit complete, ready for Phase 10.

1. What are we building?

We are building a **Flutter package** (also called a library or SDK). A package is not an app. Nobody 'opens' it on their phone. Instead, other Flutter developers add it to their own apps with one line in `pubspec.yaml`, and then they get ready-made classes and functions they can call.

Our package gives them one job, done very well: **turn invoice data into a professional PDF**, then let them preview, save, share or print it.

> **In simple words**
> Think of the package as a machine. Developers put in data (who sells, who buys, what items, which tax) and the machine gives back a finished PDF invoice. They never need to know how the machine works inside.

The developer experience we want

```
final invoice = Invoice(
  invoiceNumber: 'INV-1001',
  issueDate: DateTime.now(),
  dueDate: DateTime.now().add(const Duration(days: 30)),
  seller: Business(name: 'ABC Technologies', email: 'hello@abc.com'),
  customer: Customer(name: 'John Doe', email: 'john@example.com'),
  items: [
    InvoiceItem(name: 'Flutter App Development', quantity: 1, unitPrice: 500),
    InvoiceItem(name: 'UI/UX Design', quantity: 2, unitPrice: 100),
  ],
  tax: Tax(name: 'VAT', percentage: 13),
  discount: Discount.percentage(10),
  currency: Currency.usd,
);

final pdf = await InvoiceGenerator.generate(invoice);  // build the PDF
await InvoiceGenerator.share(invoice);                 // share sheet
await InvoiceGenerator.print(invoice);                 // print dialog
await InvoiceGenerator.save(invoice);                  // save to a file
```

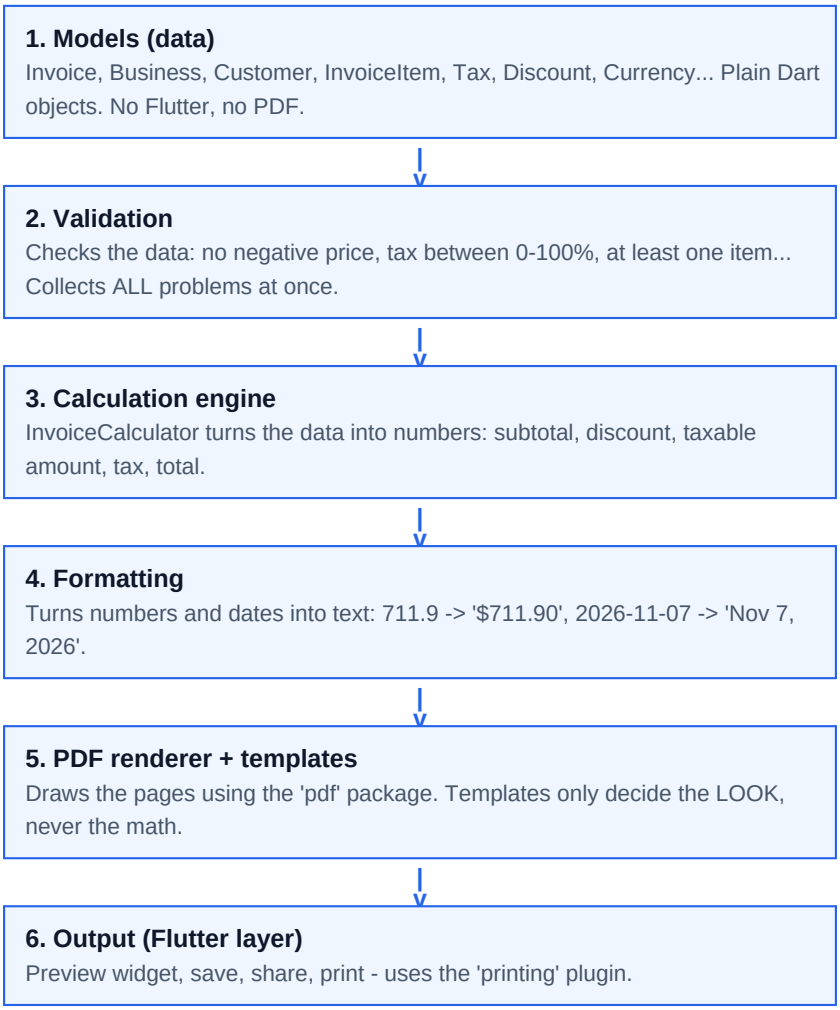
Important: your current folder

Right now `pdf_invoice/` was created with `flutter create` as an **app**. It has `android/ ios/ web/ macos/ windows/ linux/` folders and a `main.dart`. A package does not need those at the root. In Phase 2 we convert it:

- Remove the platform folders and `lib/main.dart` from the root (they are only generated boilerplate, no code of yours).
- Rename the package to `flutter_invoice_generator` in `pubspec.yaml`.
- Create a separate demo app inside `example/`. That demo app is where the platform folders belong.

2. The big picture: layers

The most important idea in this package is **separation**. Each layer does one job and only talks to the layer next to it. This keeps the code easy to test and easy to change later.



> **In simple words**
>
> It is like a restaurant. Models are the ingredients, validation is checking the ingredients are fresh, the calculator is the recipe, templates are how the plate is decorated, and output is the waiter delivering the plate. The decorator never changes the recipe.

Why this matters

- **Testing:** we can test the money math with no PDF at all - fast and exact.
- **Templates:** Classic, Modern and Minimal share the same engine. Only the drawing code differs, so a bug fixed in the math is fixed for all.
- **Server use:** layers 1-5 do not need Flutter UI, so the same code could later generate invoices on a Dart server.

3. Folder structure

This follows the structure you asked for, with a few small additions (marked NEW) that I explain below.

```
flutter_invoice_generator/
|-- lib/
|   |-- flutter_invoice_generator.dart   <- the ONLY public entry file
|   `-- src/                             <- private implementation
|       |-- models/
|       |   |-- invoice.dart
|       |   |-- invoice_item.dart
|       |   |-- business.dart
|       |   |-- customer.dart
|       |   |-- tax.dart
|       |   |-- discount.dart
|       |   |-- currency.dart
|       |   |-- payment_information.dart
|       |   `-- invoice_logo.dart          NEW: bytes / asset / network logo
|       |-- calculation/
|       |   |-- invoice_calculator.dart
|       |   |-- invoice_totals.dart
|       |   `-- money.dart                 NEW (internal): exact money math
|       |-- validation/                    NEW
|       |   `-- invoice_validator.dart
|       |-- formatting/                    NEW
|       |   `-- invoice_formatter.dart     dates + money text
|       |-- pdf/
|       |   |-- invoice_generator.dart     public facade (generate/save/...)
|       |   |-- invoice_pdf.dart           the result object
|       |   |-- invoice_pdf_builder.dart   internal engine (MultiPage)
|       |   |-- invoice_theme.dart         colors, fonts, sizes
|       |   |-- invoice_labels.dart        NEW: 'INVOICE', 'Bill To'... (translatable)
|       |   |-- sections/                  NEW: shared building blocks
|       |   |   (header, parties, items_table, summary, footer)
|       |   `-- templates/
|       |       |-- invoice_template.dart
|       |       |-- classic_template.dart
|       |       |-- modern_template.dart
|       |       `-- minimal_template.dart
|       |-- preview/
|       |   `-- invoice_preview.dart
|       |-- output/                        NEW: save / share / print
|       |   `-- invoice_output.dart
|       `-- exceptions/
|           `-- invoice_exceptions.dart
|-- assets/fonts/                         NEW: bundled Unicode font
|-- example/                             demo Flutter app
|-- test/  (models/ calculation/ validation/ pdf/)
|-- docs/learning/                       this guide (not published)
|-- README.md  CHANGELOG.md  LICENSE
|-- pubspec.yaml  analysis_options.yaml
```

Why lib/src?

In Dart, anything under `lib/src/` is considered private. Developers only import `package:flutter_invoice_generator/flutter_invoice_generator.dart`, and that file decides exactly what is visible using `export`. If a class is not exported, we can rename or rewrite it later without breaking anyone's app.

In simple words

lib/src is the kitchen. The export file is the menu. Customers only order from the menu, so we can reorganise the kitchen any time.

4. Dependencies (and why each one)

I checked pub.dev on 7 Nov 2026 for the current versions. Fewer dependencies means fewer version conflicts for the people who install us.

Package	Version	Why we need it	Could we avoid it?
pdf	^3.13.1	Builds the PDF: pages, tables, text, images, MultiPage for long invoices. Also includes 'barcode', so QR codes are free.	No - this is the core.
printing	^5.15.1	Print dialog, share sheet, PDF preview widget, loading network images. Supports Android, iOS, macOS, Windows, Linux, Web.	No - writing native print code for 6 platforms is huge.
intl	>=0.19.0 <0.21.0	Date formats ('07/11/2026') and number grouping ('1,234.50') for any locale.	Possible, but locale rules are hard. A WIDE range is used because Flutter's own localization pins one intl version.
path_provider	^2.x (to confirm)	Only for save(): finds the app documents folder on each platform.	Yes - see decision below.
meta	(already via Flutter)	@immutable annotations.	Free.

Decision for you to approve

save(): Option A (recommended): use path_provider so `save(invoice)` works with no arguments and returns the saved file path. Option B: no extra dependency, but the developer must pass a directory: `save(invoice, directory: dir)`. On Web, save() downloads the file through the browser either way.

A hidden problem: currency symbols and fonts

PDFs need a font file that contains every character we print. The built-in PDF font (Helvetica) does **not** contain the Indian Rupee sign or many non-English letters. A Nepali shop printing a customer name in Devanagari would get empty boxes.

Decision for you to approve

Bundle one open-licence font (Noto Sans, regular + bold, about 1 MB) as the default, which includes symbols like the Rupee sign. Developers can still pass any font through InvoiceTheme. This keeps generation **fully offline** (no downloading Google Fonts at runtime).

5. The models (the data)

All models are **immutable**: once created they never change. To 'edit', you call `copyWith(...)` which returns a new copy. This prevents bugs where one screen changes an invoice that another screen is still printing.

```
Invoice
 |-- invoiceNumber, issueDate, dueDate
 |-- seller   : Business   (name, email, phone, address, website,
 |                          taxId, registrationNumber, logo)
 |-- customer : Customer   (name, companyName, email, phone, address, taxId)
 |-- items    : List<InvoiceItem>
 |                (name, description, quantity (double), unitPrice,
 |                 sku, unit, tax?, discount?)
 |-- currency : Currency    (code, symbol, decimalDigits, symbolOnLeft)
 |-- taxes    : List<Tax>   (name, percentage)  <- 'tax:' shortcut for one
 |-- discount : Discount?   (percentage or fixed)
 |-- paymentInformation : PaymentInformation? (all fields optional)
 |-- notes, terms
 |-- details  : List<InvoiceDetail>  (extra rows shown, e.g. 'PO Number')
 `-- metadata : Map<String, String>  (your own data, never printed)
```

Design choices explained

- **Quantity is a double**, so 2.5 hours or 0.75 kg work.
- **Multiple taxes:** you can write `tax: Tax(...)` for the simple case or `taxes: [vat, serviceTax]`. No country rules are hard-coded - we only use the percentage you give.
- **Discount** gets friendly constructors: `Discount.percentage(10)` and `Discount.fixed(50)`. The long form `Discount(type: DiscountType.percentage, value: 10)` also works.
- **Currency** has ready constants (usd, eur, gbp, inr, npr, aud, cad, jpy) and you can make your own: `Currency(code: 'NPR', symbol: 'Rs.', decimalDigits: 2)`. JPY uses 0 decimals.
- **Logo** can come from bytes, an asset or (only if you ask) the network.
- **Optional fields everywhere** - adding a new optional field in v1.1 does not break anyone. That is how we stay backward compatible.

> **Decision for you to approve**
>
> You listed 'template configuration' as an Invoice field. I recommend **not** putting it inside Invoice. An invoice is data (what was sold); the template is presentation (how it looks). Instead: `InvoiceGenerator.generate(invoice, template: InvoiceTemplate.modern, theme: myTheme)`. Then the same invoice can be printed in any style, and saving invoices to a database never stores colours.

6. The calculation engine

Money math is where invoice software most often goes wrong. Computers store decimals like 0.1 in binary and cannot store them exactly:

```
print(0.1 + 0.2);      // 0.30000000000000004   (not 0.3!)
print(1.005 * 100);    // 100.49999999999999    (should be 100.5)
```

Our strategy: count in the smallest unit

Internally we never add doubles. We convert every amount into **whole minor units** (cents for USD, paisa for NPR, yen for JPY) stored as exact integers, do the math, and convert back only for display. We read the number through its shortest text form (1.005 stays '1.005'), so the binary error never enters the calculation. This needs **no extra package**.

> **In simple words**
> Instead of saying 'one dollar and five cents' (1.05) we say '105 cents'. Whole numbers never have rounding surprises.

The formula (worked example)

Step	Rule	Example
Line total	quantity x unitPrice - item discount	1 x 500 = 500.00 ; 2 x 100 = 200.00
Subtotal	sum of line totals	700.00
Discount	invoice discount on the subtotal	10% of 700 = 70.00
Taxable amount	subtotal - discount	630.00
Tax	each tax % of the taxable amount	VAT 13% of 630 = 81.90
Total	taxable amount + all taxes	711.90

Rounding rules (documented in code and README)

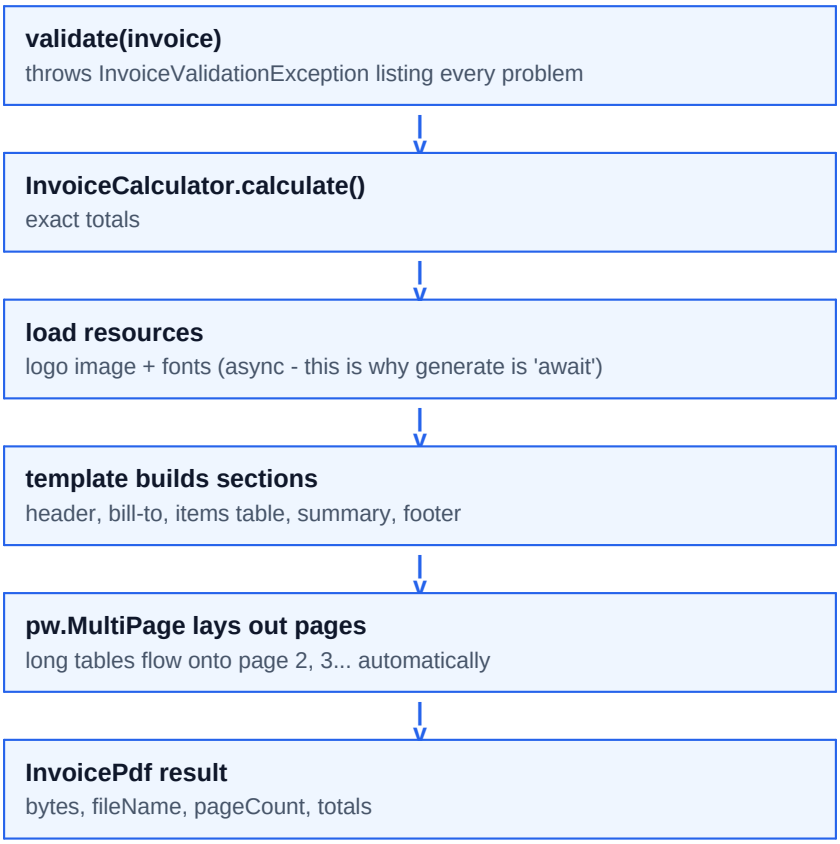
- Rounding happens to the currency's decimal places (2 for USD, 0 for JPY).
- Mode: **round half away from zero** (0.125 -> 0.13), the normal commercial rule.
- We round each line total, the discount and each tax amount once. The total is then an exact sum, so the numbers on the PDF always add up.
- If an item has its own tax, that item is taxed with its own rate; the invoice discount is shared across items in proportion to their value (leftover cents go to the lines with the largest remainder - see Chapter 3).

The result is an immutable `InvoiceTotals` object: `subtotal, discountAmount, taxableAmount, taxLines (one per tax), taxAmount, total, lines`. Developers can use it without making a PDF at all:

```
final totals = InvoiceCalculator.calculate(invoice);
print(totals.total); // 711.9
```

7. PDF generation and templates

What generate() does



What you get back

```
final pdf = await InvoiceGenerator.generate(invoice);
pdf.bytes;      // Uint8List - upload it, email it, store it
pdf.fileName;   // 'INV-1001.pdf'
pdf.pageCount;  // 3
pdf.totals;     // the same numbers printed on the PDF
```

Returning our own small `InvoicePdf` class (instead of raw bytes) means we can add fields later without breaking anyone.

Multiple pages

The pdf package has a widget called `MultiPage`. We give it a long list of content; it fills page 1, then continues on page 2. The items table header row repeats on each new page, the big business header is drawn only on page 1, and every page gets 'Page 2 of 3' in the footer. We will test 1, 10, 50 and 150 items and check the real page count.

Templates without copy-paste

Every invoice has the same five **sections**. Shared section builders live in `pdf/sections/`. A template is a small class that decides how each section looks (colours, alignment, header band or not), reusing those builders.

Template	Look
Classic	Traditional: logo left, 'INVOICE' right, bordered table, grey header row.
Modern	SaaS style: coloured header band, rounded summary card, zebra rows.
Minimal	Lots of white space, thin lines, no fills, small type.

> **Decision for you to approve**
>
> In v1, `InvoiceTemplate` exposes only the three built-in templates (`InvoiceTemplate.classic`, etc.). Opening it up for fully custom templates is planned, but doing it in v1 would freeze our internal drawing code as public API. Keeping it closed now lets us improve it freely.

Customization: InvoiceTheme and InvoiceLabels

```
InvoiceTheme(
  primaryColor: PdfColor.fromHex('#2563EB'),
  secondaryColor: PdfColors.grey700,
  font: myFont, boldFont: myBoldFont,
  baseFontSize: 10, spacing: 12, borderRadius: 6,
  pageFormat: PdfPageFormat.a4,           // or PdfPageFormat.letter
  dateFormat: 'dd/MM/yyyy', locale: 'en_US',
)
InvoiceLabels(invoiceTitle: 'TAX INVOICE', billTo: 'Bill To', ...)
```

Labels are separate so a Nepali or French developer can translate every word on the PDF without touching colours.

8. Preview, save, share, print

Preview widget

`InvoicePreview(invoice: invoice)` is a Flutter widget. Two ways to build it:

Option	Good	Bad
A. Show the real PDF (recommended)	Looks exactly like the PDF, always. Little code. Uses printing's PdfPreview.	Slightly slower to appear.
B. Re-draw it with Flutter widgets	Instant, scrollable.	Two layouts to keep in sync - they WILL drift apart over time.

Save / share / print

Platform	Print	Share	Save
Android	yes	yes (share sheet)	yes (app documents)
iOS	yes	yes (share sheet)	yes (app documents)
macOS	yes	yes	yes (needs sandbox file permission)
Windows	yes	save dialog	yes
Linux	yes	save dialog	yes
Web	browser print	browser download	browser download

This table is from the printing package docs. In Phase 7 I will verify what I can on your Mac and clearly mark anything not tested.

Error handling

```
InvoiceException                    (base - catch everything)
 |-- InvoiceValidationException     bad data. Has .issues: a list like
 |                                  ['items[2].quantity must be >= 0',
 |                                   'tax VAT: percentage must be 0-100']
 |-- InvoiceImageException          logo missing / broken / unsupported
 `-- InvoiceGenerationException     anything else while building the PDF
```

Validation collects **all** problems at once, so a developer fixes them in one go instead of one crash at a time. Rules: invoice number not empty, at least one item, quantity >= 0, price >= 0, tax 0-100%, percentage discount 0-100%, fixed discount not bigger than the subtotal, due date not before issue date.

QR codes

Not mandatory in v1. Because 'pdf' already includes the barcode library, an optional `InvoiceQrCode` (payment URL, invoice URL or custom text) costs zero new dependencies. I reserve an optional field for it and plan it for Phase 5 as a small extra.

9. The plan, phase by phase

Phase	What	You push after?
1	Architecture plan + this guide (now)	Yes - first commit
2	Convert to package, models, exports, model tests	Yes
3	Calculation engine + many tests	Yes
4	PDF generator + Classic template + multi-page tests	Yes
5	Theme, labels, Modern + Minimal templates, (QR)	Yes
6	InvoicePreview widget	Yes
7	save / share / print + platform table	Yes
8	Example app	Yes
9	README, CHANGELOG, API docs	Yes
10	pub.dev dry-run, fix everything	Yes - then publish

After every phase I stop, tell you what I did and how, add a chapter to this guide, and give you a commit message. I never commit or push - you do.

How to push (first time)

```
cd /Users/prajan/Desktop/package/pdf_invoice
git init
git add .
git commit -m "docs: add architecture plan and learning guide (phase 1)"
git branch -M main
git remote add origin https://github.com/<you>/flutter_invoice_generator.git
git push -u origin main
```

Future pro features (not in the free core)

Extra premium templates, advanced branding, custom sections, recurring invoices. The core stays a clean, free SDK - no payments or licences inside the package.

Chapter 2 - Phase 2: Skeleton and models

In this phase we turned the folder into a real package, wrote every data model, the error classes, the public export file, and 42 tests. There is no PDF code yet - that is on purpose. We first make the data solid, then build on top of it.

Step 1 - From app to package

What we did	Why
Deleted android/ ios/ web/ macos/ windows/ linux/, lib/main.dart, test/widget_test.dart	A package has no app to run. Those folders were the default counter app. The demo app will live in example/ (Phase 8).
Changed .metadata to `project_type: package`	Tells the Flutter tool this is a package, not an app.
Renamed to `flutter_invoice_generator` in pubspec.yaml	The name people will type in their pubspec. Users import `package:flutter_invoice_generator/...`.
Kept `publish_to: 'none'` for now	A safety lock so nothing gets published by accident. Removed in Phase 10.
Only one dependency added: `meta`	For `@immutable`. Flutter already ships it, so users download nothing new. pdf/printing come in Phase 4.
Stricter analysis_options.yaml	Forces doc comments on every public API, trailing commas, final locals, strict types. pub.dev scores packages on this.

Step 2 - The models

```
lib/src/models/
  currency.dart            Currency + 8 built-ins (usd, eur, gbp, inr,
                           npr, aud, cad, jpy) and Currency.values
  tax.dart                 Tax(name, percentage)
  discount.dart            Discount, DiscountType, .percentage(), .fixed()
  business.dart            seller (name required, the rest optional)
  customer.dart            buyer  (name required, the rest optional)
  payment_information.dart all optional + isEmpty
  invoice_logo.dart        InvoiceLogo.memory / .asset / .network
  invoice_detail.dart      extra header rows like 'PO Number'
  invoice_item.dart        one line
  invoice.dart             everything together
```

Five Dart ideas used everywhere (learn these!)

1. Immutable + final fields. Every field is `final` and the class is marked `@immutable`. Once made, an invoice never changes.

2. Defensive copies. If you pass a list to an Invoice and later change your list, the invoice must not change. So the constructor copies it:

```
Invoice({required List<InvoiceItem> items, ...})
    : items = List.unmodifiable(items);   // copy + read-only
```

3. copyWith. To 'edit' an immutable object you make a changed copy:

```
final updated = invoice.copyWith(invoiceNumber: 'INV-1002');
```

Note: with this common pattern, passing `null` means 'keep the old value', so copyWith cannot remove an optional field. We documented that.

4. Value equality (== and hashCode). By default Dart says two objects are equal only if they are the *same object*. We override == so two invoices with the same data are equal. The rule: if you override ==, you must override hashCode the same way, or Sets and Maps break.

5. final class. `final class Tax` means users cannot extend or implement Tax. That sounds strict, but it protects them: if we add a new field or method later, nobody's subclass breaks. This is a backward-compatibility tool.

> **In simple words**
> Immutable means 'printed on paper': you can photocopy it with changes (copyWith), but you never erase the original.

Design decisions you will see in the code

- **Numbers are `num`.** So both `quantity: 2` and `quantity: 2.5` work, and an `int` variable can be passed without `.toDouble()`. In Dart, `13 == 13.0` is true and their hashCodes match, so equality still works.
- **tax vs taxes.** Both Invoice and InvoiceItem accept `tax:` (one) or `taxes:` (many). Passing both throws an ArgumentError - in release builds too, not only in debug.
- **Item taxes have 3 meanings:** not set (null) = use the invoice taxes; a list = use these instead; `const []` = tax-exempt item.
- **Logo lives on Business.** The requirements listed a logo on both Invoice and Business. Two places for the same thing is confusing, so the seller's logo is `Business.logo`.
- **dueDate is optional** for 'due on receipt' invoices.
- **Currency defaults to USD** so the quick start stays short.
- **metadata vs details.** details are printed on the PDF (PO number); metadata is never printed (your database ID).
- **No QR field yet.** Adding an optional field later is not a breaking change, so we add it only when it actually works.

Step 3 - Sealed class for the logo

```
sealed class InvoiceLogo {
  const factory InvoiceLogo.memory(Uint8List bytes) = MemoryInvoiceLogo;
  const factory InvoiceLogo.asset(String assetPath) = AssetInvoiceLogo;
  const factory InvoiceLogo.network(String url)     = NetworkInvoiceLogo;
}
```

`sealed` means the compiler knows the full list of logo kinds. In Phase 4, when we load the logo with a `switch`, Dart will error if we forget one kind. Users only see `InvoiceLogo` - the three concrete classes are hidden with `export ... show InvoiceLogo`.

A memory logo is equal to another only if it is the same byte buffer. Comparing two 2 MB images byte by byte every time you compare invoices would be slow.

Step 4 - Exceptions

`InvoiceException` is the base class. It is deliberately **not** sealed: if it were, adding a new exception type in v1.2 would break users' exhaustive switch statements. `InvoiceValidationException` carries a list of `InvoiceValidationIssue(field, message)`, for example `items[1].quantity: must not be negative`. The validator that fills it comes in Phase 3, together with the calculator, because some rules (fixed discount bigger than the subtotal) need the totals.

Step 5 - The public door: one export file

```
// lib/flutter_invoice_generator.dart
export 'src/models/invoice.dart';
export 'src/models/invoice_logo.dart' show InvoiceLogo;
...
```

Internal helpers (`utils/equality.dart`, `utils/tax_merge.dart`) are **not** exported, so we can change them any time.

Step 6 - Tests (42 passing)

File	What it proves
test/models/currency_test.dart	8 required currencies, unique codes, JPY = 0 decimals, custom currency, equality, copyWith, bad decimals rejected
test/models/tax_discount_test.dart	decimal tax rates, 13 == 13.0, short and long Discount forms are equal, percentage != fixed
test/models/parties_test.dart	Business/Customer/Payment fields, payment without IBAN, logo equality rules
test/models/invoice_item_test.dart	quantities 1, 2, 2.5, 0.75; the 3 tax meanings; both-tax error; list copied and read-only
test/models/invoice_test.dart	quick-start invoice, defaults, multiple taxes (CGST+SGST), immutability, equality, metadata key order
test/exceptions/...	messages list every issue; all catchable as InvoiceException

Tests import only `package:flutter_invoice_generator/flutter_invoice_generator.dart`, exactly like a real user. That also proves the exports are correct.

```
flutter pub get
flutter analyze        ->  No issues found!
flutter test           ->  +42: All tests passed!
dart format lib test   ->  0 changed
```

Try it yourself: change a value in `test/models/invoice_test.dart` so a test fails, then run `flutter test`.

Next (Phase 3): InvoiceCalculator + InvoiceTotals + InvoiceValidator, the exact minor-unit money math, and many calculation tests (1 x $100, 10% discount, 13% tax, decimals, rounding, large values).

Chapter 3 - Phase 3: The calculation engine

This phase is all about money math. No PDF yet - if the numbers are wrong, a beautiful PDF is worthless. We added 4 files and 65 new tests (107 in total).

```
lib/src/calculation/
  money.dart             (internal) exact decimals, rounding, allocation
  totals_engine.dart     (internal) the math, in BigInt minor units
  invoice_totals.dart    (public)   InvoiceTotals, InvoiceLineTotals,
                                    InvoiceTaxLine
  invoice_calculator.dart(public)   InvoiceCalculator.calculate()
lib/src/validation/
  invoice_validator.dart (public)   InvoiceValidator.validate()
```

How to use it

```
final totals = InvoiceCalculator.calculate(invoice);
totals.subtotal;           // 700.0
totals.discountAmount;     // 70.0
totals.taxableAmount;      // 630.0
totals.taxLines;           // [InvoiceTaxLine(VAT: 81.9)]
totals.total;              // 711.9
totals.totalInMinorUnits;  // 71190  (cents - for Stripe etc.)
totals.lines[0].lineTotal; // 500.0
```

Idea 1 - Never do money math with double

A `double` stores numbers in binary. Most decimals (0.1, 1.005...) cannot be stored exactly, so tiny errors appear and then rounding goes the wrong way:

```
1.005 * 100           // 100.49999999999999  -> rounds to 1.00  (wrong!)
our engine: 1.005     // exactly 1005 / 1000 -> rounds to 1.01  (right)
```

How we fixed it: Dart prints every double in its shortest exact form (`1.005.toString() == '1.005'`). We read that text and turn it into two whole numbers: digits = 1005, scale = 3 (meaning 1005 / 10^3). This is `ExactDecimal` in money.dart. All math then uses `BigInt` (whole numbers of any size).

> **In simple words**
> We stop writing '1.005 dollars' and write '1005 thousandths of a dollar'. Whole numbers are always exact.

Idea 2 - Why BigInt and not int?

On Flutter Web, Dart's `int` is really a JavaScript number. It is only exact up to 2^53 (about 9 quadrillion). `quantity x price x 10^digits` can pass that for big invoices. BigInt has no limit and gives the same answer on Android, iOS, desktop and Web.

Idea 3 - Rounding: half away from zero

Exact value	Rounded (2 decimals)	Why
0.124	0.12	below half - down
0.125	0.13	exactly half - away from zero (up)
2.5 (JPY, 0 decimals)	3	not 2 - we do NOT use banker's rounding
-0.125	-0.13	away from zero, symmetric

The function is `roundDiv(numerator, denominator)`: divide, look at the remainder, and if remainder x 2 >= denominator, move one step away from zero.

Idea 4 - Round each value once, then only add

```
line   = round(quantity x unitPrice)  - round(item discount)
subtotal = sum(lines)                        <- exact, no rounding
discount = round(subtotal x %)  or fixed
taxable  = subtotal - discount               <- exact
tax      = round(taxable_for_that_rate x %)  <- once per tax rate
total    = taxable + sum(taxes)              <- exact
```

Because every 'sum' step is exact, the PDF can never show numbers that do not add up. We have a test that builds **300 random invoices** and checks this for every single one.

> **Why round tax per rate, not per line?**
> 3 lines of 0.10 at 5%: per line 0.005 rounds to 0.01, times 3 = 0.03. On the sum, 0.30 x 5% = 0.015 rounds to 0.02. Per-line rounding adds up rounding errors, so most accounting systems round tax on the total per rate. We do the same (there is a test for it).

Idea 5 - Sharing the discount fairly (largest remainder)

If items have **different** tax rates, we must know how much of the invoice discount belongs to each line. Example: discount of $1.00 over three $1 items. 100 cents / 3 = 33.33 each.

Step	Item A	Item B	Item C	Sum
Exact share	33.33	33.33	33.33	99.99?
Round down	33	33	33	99
1 cent left - goes to largest remainder (tie: first)	34	33	33	100 OK

This is `allocateProportionally()`. The shares always add up to exactly the discount. (In Chapter 1 I wrote 'leftover to last item'; largest remainder is fairer, so I changed it.)

Mixed taxes example (tested)

Item	Line	Share of 10% disc.	Taxable	Tax
A (uses invoice VAT 13%)	100	10	90	VAT 11.70
B (own GST 5%)	100	10	90	GST 4.50
C (taxes: const [] = exempt)	100	10	90	-

Item	Line	Share of 10% disc.	Taxable	Tax
Total	300	30	270	16.20 -> total 286.20

Idea 6 - The validator

`InvoiceValidator.validate(invoice)` returns a list of problems (empty = valid). `calculate()` calls it first and throws `InvoiceValidationException` if anything is wrong. It works in two rounds:

- **Round 1 - simple checks:** blank invoice number/names, no items, due date before issue date, negative or NaN quantity/price, tax not 0-100%, blank or duplicated tax, percentage discount not 0-100%, negative fixed discount.
- **Round 2 - needs the math** (only if round 1 passed, because the math would crash on NaN): fixed discount bigger than its line or the subtotal, and totals bigger than 10^15 minor units.

> **A bug the tests caught**
> An item with a $25 discount on a $20 line also made the subtotal negative, so the validator reported a second, confusing error on the invoice discount. Now it reports only the root cause. This is exactly why we write tests before building on top.

Why 10^15? Totals are returned as `double`. A double can hold any 15-digit number exactly. 10^15 cents is 10 trillion dollars - more than enough, and we can promise every value is exact.

Idea 7 - Public types you cannot construct

`InvoiceTotals` has its constructor marked `@internal`. Only our calculator should make one. That lets us add a new field in v1.1 without breaking anyone, because nobody else calls the constructor. Same idea for `abstract final class InvoiceCalculator`: it only has static methods, cannot be created, and cannot be extended.

Tests added in this phase

File	What it checks
test/calculation/money_test.dart	exact parsing (1.005, 1e-7, 1e21), half-away rounding, percent, conversion back, allocation
test/calculation/invoice_calculator_test.dart	1x$100, 2x$100, multiple items, zero qty, decimal qty/price, 0.1+0.2, 100 x 0.01, 10% / fixed / zero / 100% discount, item discounts, 13% tax, no tax, zero tax, CGST+SGST, mixed item taxes, per-rate rounding, 1.005, 0.125, JPY, 3-decimal currency, large values, 300 random invoices
test/validation/invoice_validator_test.dart	every rule above, plus 'reports every problem at once'

```
flutter analyze   ->  No issues found!
flutter test      ->  +107: All tests passed!

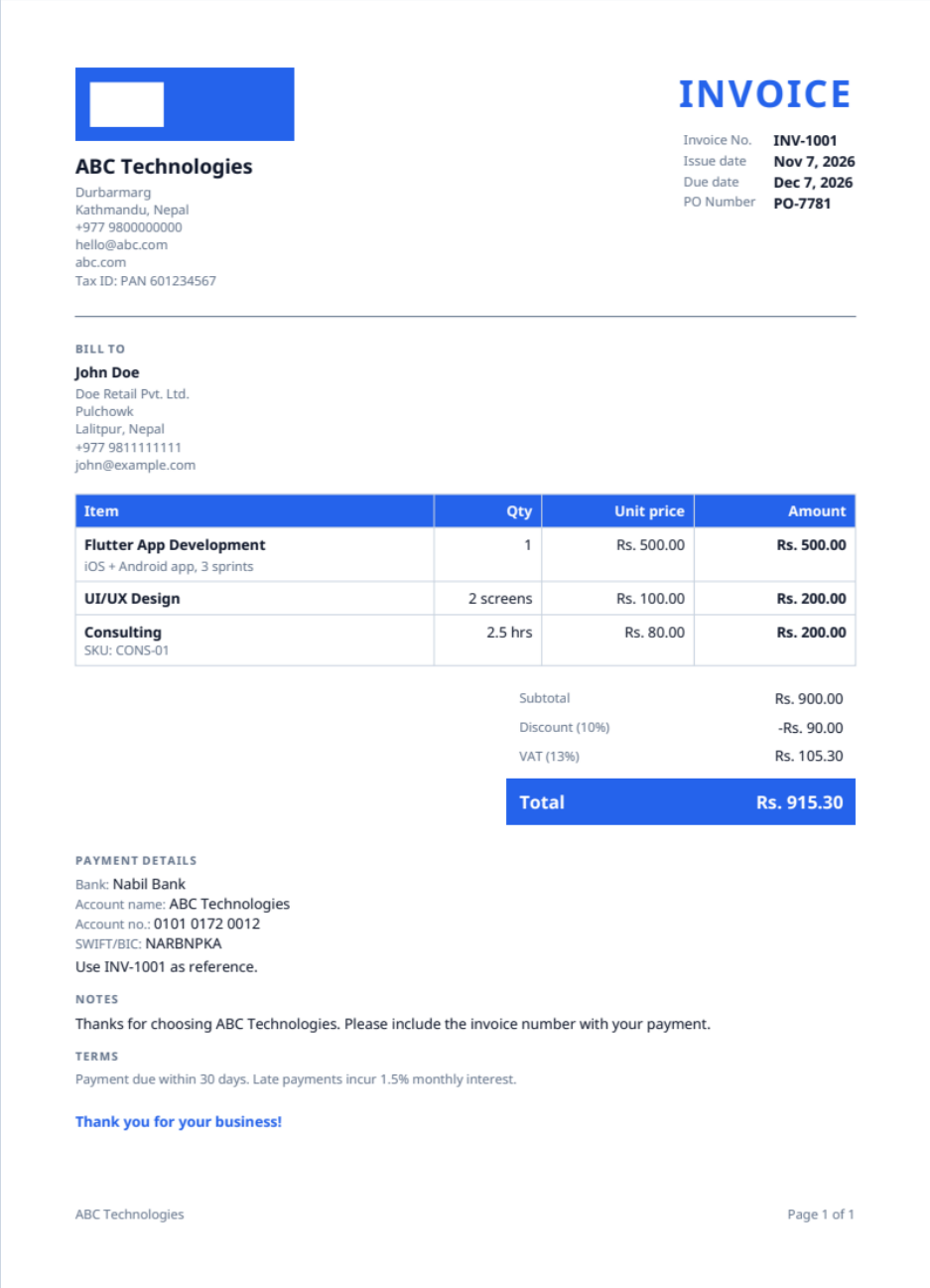
# Same tests on Web, using Brave (Chromium) instead of Chrome:
export CHROME_EXECUTABLE="/Applications/Brave Browser.app/Contents/MacOS/Brave Browser"
flutter test --platform chrome   ->  +107: All tests passed!
```

Next (Phase 4): add the `pdf` package, build `InvoiceGenerator.generate()` with the Classic template, fonts and logo loading; test 1-150 item invoices.

Chapter 4 - Phase 4: PDF generation

Now the package makes real PDFs. One line does everything:

```
final pdf = await InvoiceGenerator.generate(invoice);
pdf.bytes;      // the PDF file (Uint8List)
pdf.fileName;   // 'INV-1001.pdf'
pdf.pageCount;  // 1
pdf.totals;     // same numbers as printed
```



ABC Technologies
Durbarmarg
Kathmandu, Nepal
+977 9800000000
hello@abc.com
abc.com
Tax ID: PAN 601234567

INVOICE

Invoice No.	**INV-1001**
Issue date	**Nov 7, 2026**
Due date	**Dec 7, 2026**
PO Number	**PO-7781**

BILL TO
John Doe
Doe Retail Pvt. Ltd.
Pulchowk
Lalitpur, Nepal
+977 9811111111
john@example.com

Item	Qty	Unit price	Amount
Flutter App Development iOS + Android app, 3 sprints	1	Rs. 500.00	**Rs. 500.00**
UI/UX Design	2 screens	Rs. 100.00	**Rs. 200.00**
Consulting SKU: CONS-01	2.5 hrs	Rs. 80.00	**Rs. 200.00**

Subtotal	Rs. 900.00
Discount (10%)	-Rs. 90.00
VAT (13%)	Rs. 105.30
Total	**Rs. 915.30**

PAYMENT DETAILS
Bank: Nabil Bank
Account name: ABC Technologies
Account no.: 0101 0172 0012
SWIFT/BIC: NARBNPKA
Use INV-1001 as reference.

NOTES
Thanks for choosing ABC Technologies. Please include the invoice number with your payment.

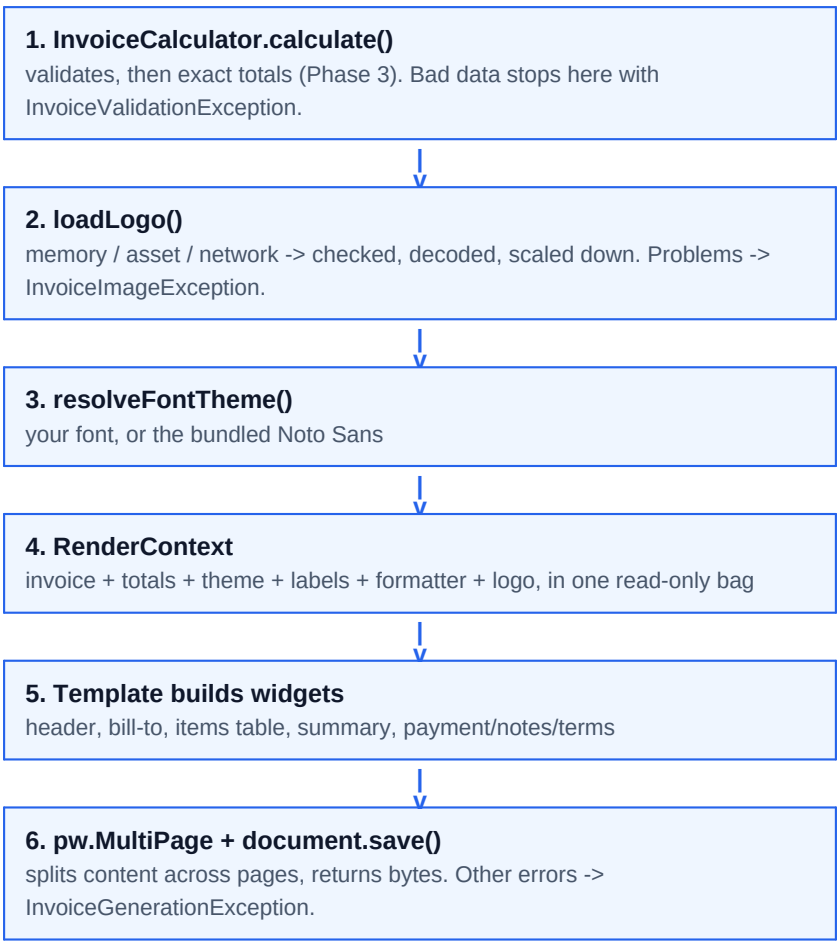
TERMS
Payment due within 30 days. Late payments incur 1.5% monthly interest.

Thank you for your business!

ABC Technologies Page 1 of 1

The Classic template, generated by the package (NPR currency, logo, payment details, notes, terms).

What happens inside generate()



New files

```
lib/src/formatting/invoice_formatter.dart   (public) money/date/qty text
lib/src/pdf/
  invoice_generator.dart      (public)   InvoiceGenerator.generate()
  invoice_pdf.dart            (public)   the result
  invoice_theme.dart          (public)   page size, colours, fonts, formats
  invoice_labels.dart         (public)   every word (translatable)
  render_context.dart         (internal)
  resources/font_loader.dart  (internal) bundled font, cached
  resources/logo_loader.dart  (internal) logo checks + scaling
  sections/                   (internal) shared building blocks:
    party_section.dart  items_table.dart  summary_section.dart
    footer_section.dart
  templates/
    invoice_template.dart     (public)   InvoiceTemplate.classic
    template_renderer.dart    (internal) the contract
    classic_template.dart     (internal) the Classic look
assets/fonts/  NotoSans-Regular.ttf  NotoSans-Bold.ttf  OFL.txt
```

Step 1 - Dependencies, chosen carefully

Before writing code I read the real source of each package in `~/.pub-cache` instead of trusting memory. Findings that changed the design:

Finding	What we did
The newest pdf (3.13) and printing (5.15) need Dart 3.12 - released this year.	Used `pdf: ^3.12.0`. Apps on older Flutter can still install us; new apps still get 3.13.
printing.networkImage() returns a finished image, skipping our checks.	Download logos with `http` instead (already inside printing). printing is added in Phase 6, when it is really needed.
MultiPage has `maxPages = 20` by default.	Raised to 1000, tested with a 1000-item invoice.
`TableRow(repeat: true)` exists.	Header row repeats on every page automatically.
No TooManyPagesException class in this version.	Catch everything and wrap it in InvoiceGenerationException.

Step 2 - The font problem, solved offline

The built-in PDF fonts (Helvetica...) cannot draw the Rupee sign. I downloaded Noto Sans (free OFL licence), and **checked** with a font tool that it really contains $ EUR GBP INR JPY and the Rupee signs before adding it. The licence file is shipped next to the fonts.

- Fonts are ~600 KB each in the app, but the PDF only embeds the letters actually used (subsetting): our sample PDF is just 19 KB.
- Font bytes are cached after the first load (read-only data, not user state), so the second invoice is faster.
- Noto Sans has no Devanagari. For Nepali/Hindi text pass a font in `InvoiceTheme(fontFallback: [devanagariFont])`.

Step 3 - Logos that fail clearly

Input	What happens
null	no logo, no error
Small JPEG (<= 1 MB)	embedded as-is: fastest, no quality loss
PNG / GIF / BMP / WebP / TIFF / big JPEG	decoded now; longest side scaled to 1024 px (a 3000 px photo would bloat the PDF)
Empty, random bytes, TGA	InvoiceImageException: 'not recognised... Supported formats: ...'
Missing asset	InvoiceImageException mentioning pubspec.yaml
Network 404 / offline / bad URL	InvoiceImageException with the reason, suggesting memory/asset logos

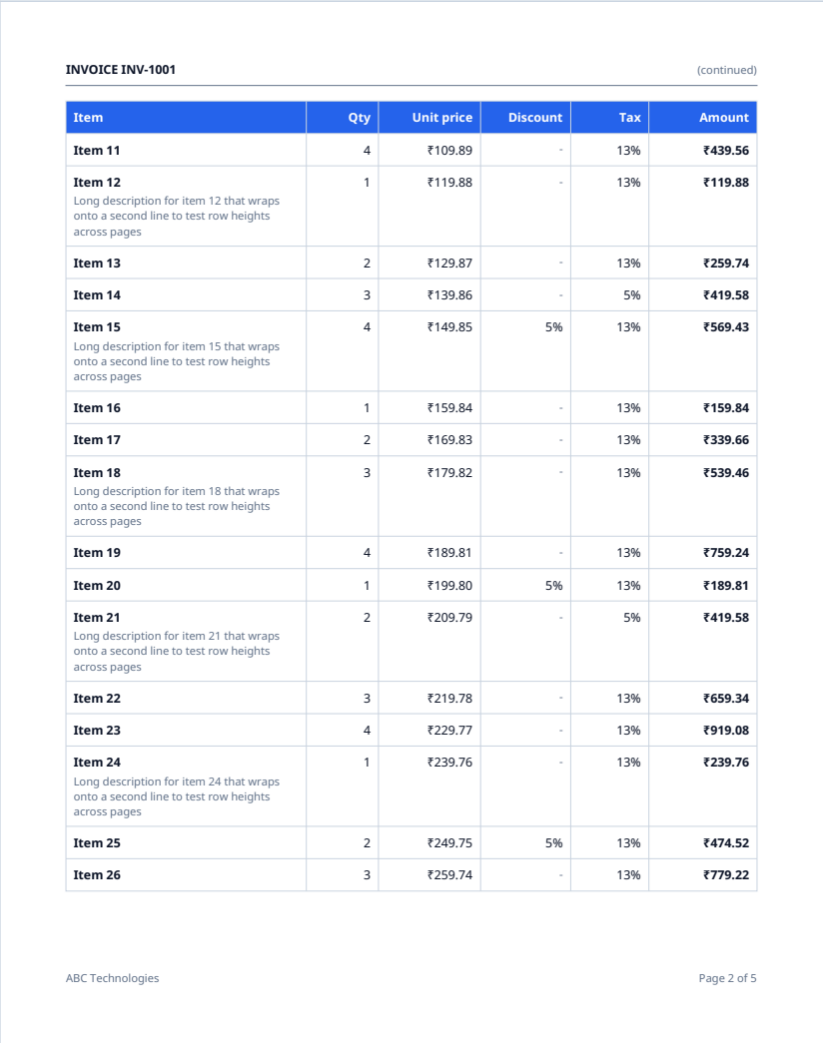
Why decode early? The pdf library decodes PNGs only while saving. A corrupt file would then fail deep inside the library with a confusing message. Decoding first gives the developer a clear error.

Step 4 - Templates without copy-paste

The **sections** (party block, items table, summary, closing sections, page header/footer) are shared functions. A template only composes them and passes a style - for example `ItemsTableStyle(headerBackground: primaryColor, border: ...)`. Modern and Minimal (Phase 5) will reuse the same sections with different styles. The public `InvoiceTemplate`

is just a handle; the drawing code stays internal, so we can improve it freely.

Step 5 - Multi-page done right



INVOICE INV-1001 (continued)

Item	Qty	Unit price	Discount	Tax	Amount
Item 11	4	₹109.89	-	13%	**₹439.56**
Item 12 Long description for item 12 that wraps onto a second line to test row heights across pages	1	₹119.88	-	13%	**₹119.88**
Item 13	2	₹129.87	-	13%	**₹259.74**
Item 14	3	₹139.86	-	5%	**₹419.58**
Item 15 Long description for item 15 that wraps onto a second line to test row heights across pages	4	₹149.85	5%	13%	**₹569.43**
Item 16	1	₹159.84	-	13%	**₹159.84**
Item 17	2	₹169.83	-	13%	**₹339.66**
Item 18 Long description for item 18 that wraps onto a second line to test row heights across pages	3	₹179.82	-	13%	**₹539.46**
Item 19	4	₹189.81	-	13%	**₹759.24**
Item 20	1	₹199.80	5%	13%	**₹189.81**
Item 21 Long description for item 21 that wraps onto a second line to test row heights across pages	2	₹209.79	-	5%	**₹419.58**
Item 22	3	₹219.78	-	13%	**₹659.34**
Item 23	4	₹229.77	-	13%	**₹919.08**
Item 24 Long description for item 24 that wraps onto a second line to test row heights across pages	1	₹239.76	-	13%	**₹239.76**
Item 25	2	₹249.75	5%	13%	**₹474.52**
Item 26	3	₹259.74	-	13%	**₹779.22**

ABC Technologies Page 2 of 5

Page 2 of a 60-item invoice: small 'continued' header, repeated table header, page numbers. Discount and Tax columns appear only because some items have them.

- Page 1 has the big header; pages 2+ get 'INVOICE INV-1001 (continued)'.
- Every page: seller name and 'Page X of Y'.
- Columns are smart: Discount only if an item has its own discount; Tax only if an item has its own tax (otherwise the rate is in the summary).

> **Two layout bugs found by LOOKING at the output**
>
> 1) A 3-item invoice used 2 pages: only 'Thank you' fell onto page 2. Fix: the thank-you line is glued to the last section so it can never be alone. 2) The spacing was too generous; tightened so a full invoice fits on one page. Tests passed in both cases - always look at the real PDF too!

Step 6 - Testing a PDF for real

A PDF compresses its text. Our test helper `PdfInspector` un-zips the page streams with dart:io's zlib and pulls out every printed word. With a standard font words are stored as `[(Word)]TJ`, so tests

use Helvetica to be able to read them. Now tests can check:

- The invoice number, dates, names, $711.90 total, 'Page 1 of 1' are printed.
- 1, 10, 50 and 150 items: every item printed exactly once; the header 'Qty' printed once **per page** (proves it repeats); 'continued' on pages 2+.
- 1000 items work (past pdf's default 20-page limit).
- A4 = 595 x 842 points, Letter = 612 x 792 (read from the MediaBox).
- All 8 currencies print with no 'Unable to find a font' warning - and a second test proves the check works (Helvetica + Rupee does warn).
- Logos: 5 formats, scaling, empty/garbage/TGA/truncated, asset missing, network OK/404/offline (fake HTTP client).

> **My own test had the floating-point bug!**
> I first wrote `expect(total, count * 10 * 0.9 * 1.13)`. That gives 10.169999999999998 - the package correctly returned 10.17. The fixed test compares whole cents. Even experienced developers make this mistake.

Step 7 - Honest version ranges (pub downgrade)

`flutter pub downgrade` installs the **oldest** versions our pubspec allows. That found: image 4.1.0 pulls an archive version that no longer compiles on modern Dart, so the lower bound became `image: '>=4.3.0'`. It also found that old image versions accept random bytes as 'TGA', so we now accept only formats with a real signature. Result: our code works with pdf 3.12.0, http 0.13.0, intl 0.19.0 and with the newest versions.

Known upstream issue: pdf 3.12.0 + image 4.11.0 is broken (image's own bug, fixed in 4.11.1, which needs pdf 3.13). Normal installs never pick it; it is noted for the README troubleshooting section.

Locale bug fixed

The formatter used to load date data immediately, so even money formatting failed for 'de_DE' when German date data was not loaded. Now dates load lazily, unknown locales say 'not supported', and missing date data says exactly what to call: `initializeDateFormatting("de_DE")`.

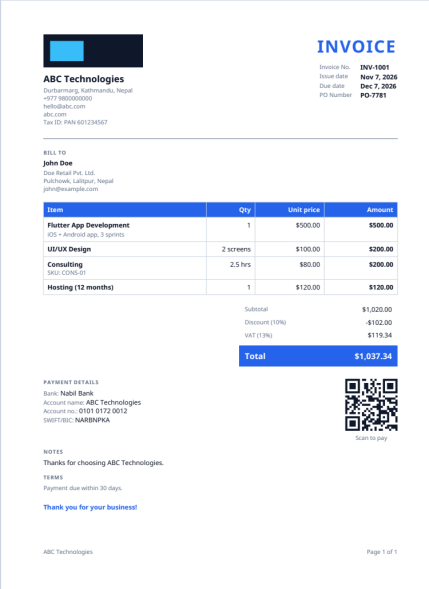
```
flutter analyze                 ->  No issues found!
flutter test                    ->  +164: All tests passed!
flutter test --platform chrome  ->  +119 (pure-Dart tests, Brave)
```

Next (Phase 5): Modern and Minimal templates, more theme options (spacing, radius, table style, header/footer), optional QR code.

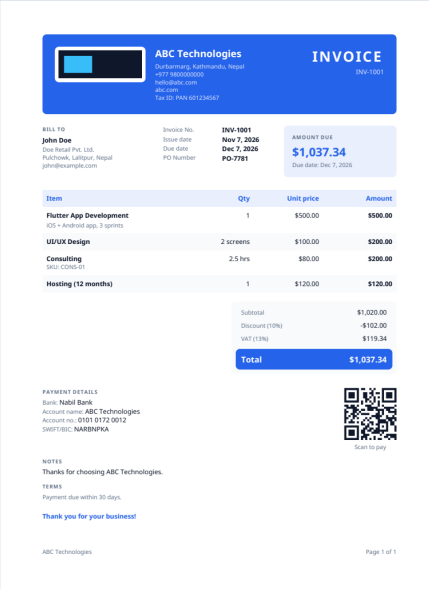
Chapter 5 - Phase 5: Templates and customization

Same invoice, three looks. Only one argument changes:

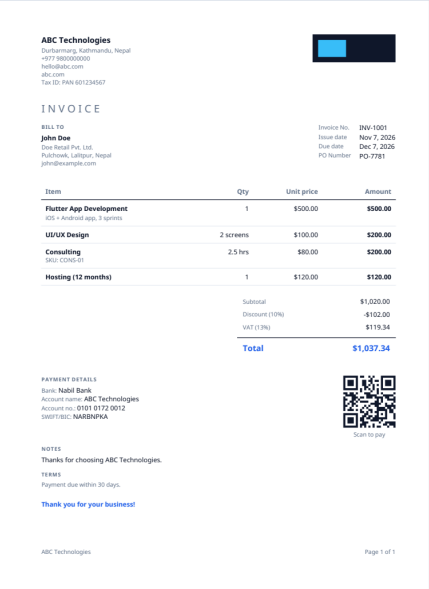
```
await InvoiceGenerator.generate(invoice, template: InvoiceTemplate.classic);
await InvoiceGenerator.generate(invoice, template: InvoiceTemplate.modern);
await InvoiceGenerator.generate(invoice, template: InvoiceTemplate.minimal);
```



Classic - grid table, coloured header



Modern - colour band, amount-due card, stripes



Minimal - no fills, hairlines, white space

How 3 templates share one engine

Every template implements the same small internal contract:

```
abstract interface class TemplateRenderer {
  List<pw.Widget> buildContent(RenderContext ctx);   // the page body
  pw.Widget buildPageHeader(RenderContext ctx, pw.Context page);
  pw.Widget buildPageFooter(RenderContext ctx, pw.Context page);
}
```

A template is ~100-150 lines that **arrange shared sections** with different styles. It never calculates, validates, loads fonts or handles pages - the engine does that once for everybody:

Shared section	Classic	Modern	Minimal
itemsTable	grid, blue header	tinted header, stripes, no lines	no fill, dark underline, hairlines
summaryBox	filled total row	grey card + rounded total	line above total, coloured text
partyBlock / metaTable	header right side	3-column info row	bill-to left, meta right
closingSections (+QR)	same	same	same

Shared section	Classic	Modern	Minimal
page header / footer	same	same	same

> **In simple words**
> Like one kitchen with three plate designs: the food (data and totals) is identical, only the plating changes. A test proves it: all three templates report exactly the same total, 71190 cents.

Customization: InvoiceTheme

```
InvoiceTheme(
  primaryColor: PdfColor.fromHex('#16A34A'),   // accent colour
  secondaryColor: ..., textColor: ...,
  font: myFont, boldFont: myBold, fontFallback: [devanagari],
  baseFontSize: 10,            // all text sizes scale from this
  sectionSpacing: 14,          // NEW: space between sections
  borderRadius: 6,             // NEW: rounded corners (Modern)
  tableStyle: InvoiceTableStyle(   // NEW: override the table look
    headerColor: ..., headerTextColor: ..., stripeColor: ...,
    borderColor: ..., showBorders: false, rowPadding: 6),
  showLogo: true,              // NEW
  showPageNumbers: true,       // NEW
  showContinuationHeader: true,// NEW
  footerText: 'Confidential',  // NEW (default: seller name)
  pageFormat: PdfPageFormat.letter,
  locale: 'en_US', dateFormat: 'dd/MM/yyyy',
)
```

Template defaults + theme overrides: each template has its own table look. `InvoiceTableStyle` fields are all nullable - `null` means 'keep the template's choice'. Internally `ItemsTableStyle.resolve()` merges them with ??.

> **Backward compatible on purpose**
> Every new theme field is optional with a default, and the QR code is a new optional Invoice field. Code written for Phase 4 still compiles and produces the same PDFs. That is how you add features to a package without a breaking release.

QR codes - zero new dependencies

```
Invoice(..., qrCode: InvoiceQrCode.payment('https://pay.example.com/1001'))
InvoiceQrCode.invoiceUrl('https://example.com/i/1001')  // 'Scan to view online'
InvoiceQrCode(data: 'any text', caption: 'Our website')
```

- The pdf package already re-exports the barcode library, so `pw.BarcodeWidget(barcode: pw.Barcode.qrCode())` costs nothing extra.
- The QR is printed beside the payment details in every template.
- Validation: empty data, or data too long for a QR code (`Barcode.qrCode().isValid()`), gives an InvoiceValidationException on field `qrCode.data`.
- Captions come from InvoiceLabels (scanToPay, scanToView), so they translate with everything else.

Two bugs found this phase

1. Transparent colours print solid in PDF fills

Modern's 'amount due' card used the primary colour with 8% alpha. PDF fills ignore alpha, so it printed solid blue and the text disappeared. Found by looking at the rendered page. Fix: mix the colour with white ourselves: `1 - (1 - c) x 0.1` per channel.

2. An old library version crashes while probing tiny data

Running the tests at the oldest allowed versions (pub downgrade) showed image 4.10.2's format detection throws a RangeError on a 4-byte 'logo'. It escaped as a generic error. Fix: the detection itself is now inside try/catch, so it is always an InvoiceImageException.

How we test looks without screenshots

- **Colours:** the theme uses #336699 = (0.2, 0.4, 0.6). The test checks the PDF drawing commands contain `0.2 0.4 0.6 rg` ('rg' = set fill colour) for every template and for tableStyle overrides.
- **Toggles:** no 'Page 1 of 1' when showPageNumbers is false; 'Confidential' once per page with footerText; no '(continued)' when disabled; no image object when showLogo is false.
- **Every template:** key text present, 150 rows each printed once, header repeated per page, identical totals.
- **QR:** captions, drawn in all templates (many extra drawing commands), invalid data rejected.

```
flutter analyze                 ->  No issues found!
flutter test                    ->  +188: All tests passed!
pub downgrade (oldest versions)  ->  +188: All tests passed!
```

Next (Phase 6): the InvoicePreview Flutter widget (shows the real PDF, so it always matches), adding the printing package.

Chapter 6 - Phase 6: The preview widget

Developers can now show the invoice inside their app with one widget:

```
InvoicePreview(
  invoice: invoice,
  template: InvoiceTemplate.modern,     // optional
  theme: myTheme,                       // optional
  allowPrinting: true,                  // optional toolbar buttons
  allowSharing: true,
  onGenerated: (pdf) => setState(() => lastPdf = pdf),
  errorBuilder: (context, error) => Text(error.message),
)
```

Decision: show the real PDF, not a copy of it

In Phase 1 we chose option A: render the **actual PDF bytes** with printing's `PdfPreview`. The preview can never differ from the file the customer receives, and there is no second layout to maintain. This is also why `printing` was only added now - this is the first phase that needs it.

> **Architecture kept clean**
> Only `lib/src/preview/invoice_preview.dart` imports Flutter's material library. Models, calculator, validator and the PDF engine still know nothing about widgets:
> Invoice -> Calculator -> PDF renderer -> bytes -> (optional) InvoicePreview

Reading the source first saved us from flicker

Before writing the widget I read `PdfPreview`'s `didUpdateWidget`:

```
if (oldWidget.build != widget.build || widget.shouldRepaint || ...) {
  // throw away the rendered pages and render again
}
```

It compares the `build` **function object**. A closure written inline in `build()` is a new object every rebuild, so the preview would re-render (and flash) every time the parent rebuilt - for example on every keystroke in a form. Our fix: generate the PDF ourselves and create the closure only when new bytes arrive:

```
setState(() {
  _pdf = pdf;
  _build = (_) async => bytes;   // new object ONLY when the PDF changed
});
```

When does it regenerate? Identity vs equality

Change	Regenerates ?	Why
Parent rebuilds with an equal invoice/theme	No	values compared with == (we added == to InvoiceTheme, InvoiceLabels, InvoiceTableStyle this phase)

Change	Regenerates ?	Why
Invoice data, template, theme or labels change	Yes	
allowPrinting / maxPageWidth / colours of the frame	No	presentation of the preview only

In simple words

Identity asks 'is it the same box?'. Equality asks 'is the same thing inside?'. Widgets are rebuilt constantly, so we must compare what is inside, otherwise we would redo expensive work all the time.

Lifecycle of the State

```
initState()            -> _generate()
didUpdateWidget(old)   -> if inputs changed: _generate()
_generate():
  final generation = ++_generation;          // ticket number
  pdf = await InvoiceGenerator.generate(...);
  if (!mounted || generation != _generation) return;   // stale -> drop
  setState(...)                               // show it
```

- **mounted check:** the user may leave the screen before the PDF is ready; calling setState then would throw.
- **Ticket number:** if the user edits again while a PDF is being built, the older result arrives later and is dropped, so the screen can never jump back to an older version.
- **Previous PDF stays visible** while the next one is generated (no blank flash); the spinner only appears the very first time.
- **Errors are UI, not crashes:** invalid data shows the list of problems (or your errorBuilder), and fixing the data brings the preview back.

A widget-testing lesson: fake time vs real time

Widget tests run in a **fake-async zone**: timers do not really wait. PDF generation does real work, so the tests wrap waiting in `tester.runAsync()`. The first test run still failed: the PDF was generated (the callback fired) but the screen showed the spinner.

Why: microtasks run after the frame

The generation started in initState, inside the fake zone. Its last step (setState) ran as a microtask *during* `tester.pump()` - after that frame had already been built. Nothing drew the new state. Fix: pump one more frame after the event. The widget was correct; the test helper was not.

Two more tests were wrong, not the widget: one expected a regeneration that correctly never happened (equal invoice), and one assumed generations would overlap, but with cached fonts each finished before the next edit. Lesson: when a test fails, first ask **which side is wrong** - the code or the test.

Tests added (12)

- Loader first, then PdfPreview; the bytes shown equal the generated PDF.
- Toolbar hidden by default; enabling printing does not regenerate.
- Invalid data -> default error lists 'items[0].quantity: must not be negative'; custom loading/error builders are used.
- Equal inputs -> no regeneration and the *same* build function (proves no flicker).
- Invoice, template, theme changes each regenerate; quick edits end on the latest; recovery after fixing an error.
- Value equality of InvoiceTheme (fonts by identity), InvoiceLabels, InvoiceTableStyle.

```
flutter analyze                  ->  No issues found!
flutter test                     ->  +200: All tests passed!
oldest versions (printing 5.14.3, pdf 3.12.0)  ->  +200 passed
```

Not verifiable in unit tests: the page images themselves (they are drawn by the native printing plugin). We check them visually in the example app in Phase 8.

Next (Phase 7): InvoiceGenerator.save / share / print, with an honest per-platform support table.

Chapter 7 - Phase 7: Save, share and print

```
await InvoiceGenerator.print(invoice);              // print dialog
await InvoiceGenerator.share(invoice);              // share sheet
final saved = await InvoiceGenerator.save(invoice);  // file
print(saved.path);    // null on the web (browser download)

// Reuse an existing PDF (no second generation):
final pdf = await InvoiceGenerator.generate(invoice);
await pdf.print();  await pdf.share();  await pdf.save(directory: dir);
```

Design: two ways in, one implementation

- **Extension methods** on InvoicePdf (`extension InvoicePdfActions on InvoicePdf`) do the real work. An extension adds methods to a class without changing the class - InvoicePdf itself stays a plain data object.
- **Static shortcuts** InvoiceGenerator.print/share/save = generate + the extension. Simple for beginners, efficient for apps that already have the PDF (for example from InvoicePreview.onGenerated).
- **InvoiceSaveResult** instead of returning a String: today it has path, fileName and isDownload; tomorrow we can add fields without breaking anyone.
- **InvoiceOutputException** (new subclass): platform failures are different from bad data or a broken logo. Because InvoiceException is not sealed, adding it was not a breaking change - exactly the reason we chose that in Phase 2.

Research first: what does 'share' really do?

Instead of trusting the plugin's marketing, I read its native code for every platform:

Platform	share() actually...	Found in
Android	opens the share chooser	PrintingJob.java: Intent.createChooser
iOS	opens the share sheet	PrintJob.swift: UIActivityViewController
macOS	opens the share menu	PrintJob.swift: NSSharingServicePicker
Windows	**writes a temp file and opens it** in the PDF viewer	print_job.cpp: ShellExecuteEx 'open'
Linux	**writes /tmp/... and runs xdg-open**	print_job.cc: execlp
Web	downloads the file	printing_web.dart: link.download

So the README says honestly that Windows/Linux have no share sheet. We also learned macOS printing needs the `com.apple.security.print` entitlement.

save(): path_provider + a web-safe conditional import

As decided in Phase 1, `save()` uses `path_provider` (Flutter team) to find the app documents folder. Writing a file needs `dart:io`, which does not exist in browsers - and pub.dev marks a package that imports it directly as 'not web compatible'. Solution:

```
// file_writer.dart
export 'file_writer_web.dart' if (dart.library.io) 'file_writer_io.dart';
```

Mobile/desktop builds get the dart:io version; web builds get a stub that makes `save()` fall back to a browser download. A browser test (run in Brave) proves the web path works.

> **Honest about 'saved where?'**
> The app documents folder is **private** on Android and (by default) on iOS - the user will not find the file in their Files app. The docs say so, and suggest share() when the user should choose the location (the share sheet has 'Save to Files').

Testing plugins without a phone

Plugins talk to native code, which does not exist in `flutter test`. Federated plugins expose a replaceable **platform interface**, so tests install fakes:

```
PrintingPlatform.instance = FakePrinting();          // records calls
PathProviderPlatform.instance = FakePathProvider(tempDir);
```

> **A fake that works with two plugin versions**
> printing 5.15 added a parameter to layoutPdf(). A fake that *overrides* layoutPdf compiles with only one version. Our fake overrides `noSuchMethod` instead: any call is caught, recorded, and answered - so the same test works with printing 5.14.3 and 5.15.1.

- print: right PDF bytes reach the dialog, job name, page format (Letter), no re-layout, cancel returns false.
- share: bytes, file name, subject, text, emails; an existing PDF is shared twice without being regenerated (same bytes object).
- save: exact bytes written, nested folders created, custom name, replaces existing, default = documents dir, unwritable path -> InvoiceOutputException.
- MissingPluginException -> message explaining hot restart vs full restart (the #1 beginner plugin mistake).
- Invalid invoice -> validation error and **no** dialog opened, no file written.

pub downgrade found another ecosystem trap

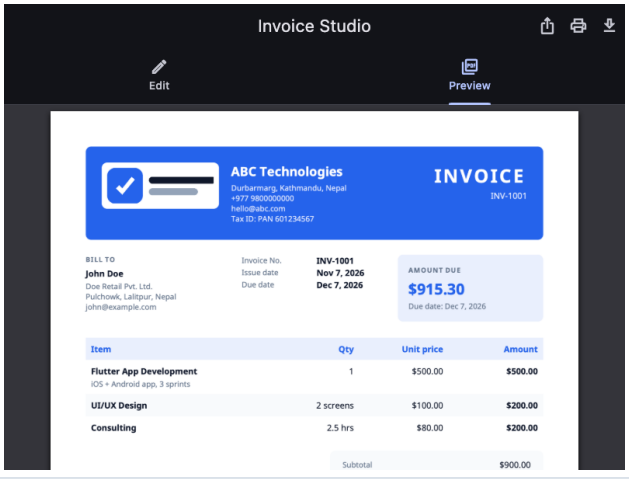
At the oldest versions, path_provider's interface pulled `platform 3.0.0`, which uses an API removed from Dart. Every interface version allows it, so no bound of ours can prevent it; normal installs always get platform 3.2.0. For the check we pinned platform 3.1.4 and confirmed **our** minimums (pdf 3.12.0, printing 5.14.3, path_provider 2.1.0, http 0.13.0, intl 0.19.0) pass all tests.

```
flutter analyze  -> No issues  |  flutter test -> +214 passed
web (Brave) -> +121 passed     |  oldest versions -> +214 passed
```

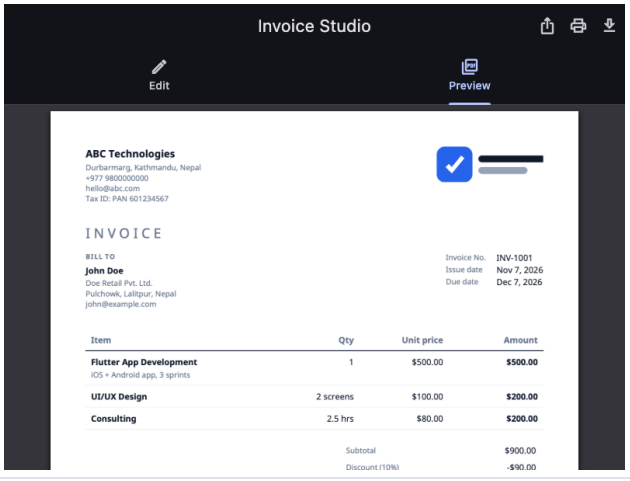
Not verified yet: the real native dialogs - checked by hand on macOS in the example app. **Next (Phase 8):** the example app.

Chapter 8 - Phase 8: The example app

Invoice Studio (`example/`) is the package's demo. pub.dev shows it on the package page, and developers copy from it, so it must be correct and readable - not only pretty.



Real macOS app, Modern template. The page image is drawn by the native printing plugin.



After tapping 'Minimal' in the editor: the preview updated.

Structure

```
example/
  pubspec.yaml              flutter_invoice_generator: path: ../
  assets/logo.png           demo logo (InvoiceLogo.asset)
  lib/main.dart             MaterialApp, light + dark theme
  lib/src/invoice_draft.dart   mutable form state -> toInvoice()
  lib/src/editor_panel.dart    form sections (style, business, customer,
                               items, discount & tax, details & payment)
  lib/src/home_page.dart       layout, debounce, preview, actions
  test/widget_test.dart        7 tests (draft + layouts + validation)
  integration_test/app_test.dart  runs inside the real app
  macos/Runner/*.entitlements  + print, + network client
```

Key idea: a mutable draft feeding immutable models

The package's models are immutable (Chapter 2) - perfect for data, awkward for a form where the user edits one field at a time. The app keeps a small `InvoiceDraft extends ChangeNotifier` with plain mutable fields and converts it on demand:

```
draft.update(() => draft.customerName = value);   // form changes the draft
final invoice = draft.toInvoice();                 // package model, immutable
```

> **In simple words**
>
> The draft is the notepad you scribble on; toInvoice() writes a clean copy on official paper. The package only ever sees the clean copy.

UX details worth copying

- **Debounce:** the preview updates 350 ms after the user stops typing (a Timer that is reset on every change), so the PDF is not rebuilt on every keystroke.
- **Reuse the PDF:** InvoicePreview.onGenerated stores the PDF; Share/Print/Save call `pdf.share()` etc. - no second generation, and what you share is exactly what you see.
- **Validation from the package:** a banner lists `InvoiceValidator.validate()` issues; the form itself does not duplicate the rules.
- **Responsive:** 960 px or wider = editor | preview side by side; narrower = Edit / Preview tabs (editor kept alive when switching).
- **Stable keys** on item rows (ValueKey(item.id)) so deleting item 1 does not move its text into item 2.
- **iPad share:** the Share button passes its own rectangle as sharePositionOrigin.

Bugs found while building it

Phone overflow (found by a widget test)

At 400 px wide the currency dropdown overflowed by 23 px. Fix: `isExpanded: true`. The test sets `tester.view.physicalSize` to a phone size - an easy way to test small screens.

Using BuildContext after await (found by the analyzer)

The error colour was read with Theme.of(context) *after* an await. If the screen closed meanwhile, that context is dead. Fix: read messenger and colours before the first await.

Integration test: proof on a real platform

Unit tests fake the plugins. An **integration test** runs inside the real macOS app with the real plugins:

```
cd example && flutter test integration_test -d macos
```

- Waits until the native plugin has drawn page images inside InvoicePreview, then saves a screenshot (`RenderRepaintBoundary.toImage()`) - the pictures above.
- Taps 'Minimal', waits for the debounce, checks the preview re-renders.
- `save()` wrote `~/Library/Containers/<app id>/Data/Documents/INT-TEST-1.pdf` - exactly the sandbox path in our README table.

Debugging by asking the screen

The first run timed out. Instead of guessing, the test printed what was on screen: the 'Edit / Preview' tabs. The test window is 800 px, below the 960 px breakpoint, so the app had correctly chosen the phone layout - the preview was on the hidden tab. The fix was in the test (tap 'Preview'), not in the app.

Still by hand: the real print dialog and share menu are modal system windows that tests cannot click. You verify them by running the app (see the end-of-phase message).

```
package tests 214 passed | example tests 7 passed | integration (macOS) 2 passed
```

Chapter 9 - Phase 9: Documentation

For a package, documentation *is* the product's front door: pub.dev shows the README as the package page, and developers decide in about two minutes whether to use it. This phase added or rewrote four things:

File	What it is for
README.md	The pub.dev page: overview, install, quick start, complete example, templates, customization, tax, discounts, currency, calculations, PDF, multi-page, preview, save/share/print, platform table, errors, FAQ, troubleshooting, roadmap
CHANGELOG.md	What changed in each version (Keep a Changelog style). pub.dev shows it in a 'Changelog' tab
LICENSE	MIT for the code + note that the fonts are under the OFL
dartdoc comments	The API reference (/// comments) - checked with dart doc: 0 warnings, 0 errors

How the README is organised

- **First screen:** one line of code, six check-marks, three template pictures. A reader knows what it does before scrolling.
- **Quick start is tiny** (5 required fields), then a **complete example** that uses almost every option - easy to copy and delete what you do not need.
- **Honesty sections:** the platform table says how each cell was verified, and the troubleshooting section lists real problems we hit while building (hot restart, macOS entitlement, date locales, fonts, CORS, private folders, an upstream image/pdf bug).

Docs that cannot lie: test/readme_test.dart

Every README snippet is repeated in a test. If someone changes the API and forgets the README, a test fails. It also checks the numbers written in README comments (700, 711.9, 915.3, 'Rs. 1,234.50', ...).

It caught two real mistakes

1. A wrong number in the README: the complete example said `// 1037.34`, copied from an older sample with an extra 'Hosting' item. The real total is 915.30.

2. A real package bug: `InvoiceFormatter.money(1.2345)` printed KD1.234 while the calculator says 1.235. The formatter gave the raw double to intl, which rounds the binary value 1.23449999... On the PDF this meant a unit price of 1.005 showed '$1.00' next to a line total of '$1.01'. Fix: the formatter now rounds with the same exact decimal code as the calculator.

In simple words

Writing the documentation is also a final code review: explaining a feature in plain words forces you to try it the way a new user would.

Choosing a license

MIT lets anyone use the package, including in commercial apps, as long as they keep the copyright notice. That fits the plan: a free, popular core, with possible paid extras (premium templates, support) sold separately - not inside this package. The bundled fonts keep their own SIL Open Font License, which is shipped next to them.

What is still a placeholder

- Image links in the README use `OWNER/REPO`; they need your GitHub repository (pub.dev cannot show relative images).
- The copyright holder name in LICENSE.
- Manual check of the native print dialog / share menu on macOS.

```
flutter analyze -> No issues | flutter test -> +224 | dart doc -> 0 warnings
```

Next (Phase 10): pubspec metadata (repository, topics, screenshots), .pubignore, version 0.1.0, and `dart pub publish --dry-run` until it is clean.

Chapter 10 - The final audit

Before publishing, we stopped adding features and tried to **break** the package: unusual inputs, a second read of every file, and every check from earlier phases again. All 224 tests were passing - and we still found a serious bug.

1. Stress testing: the freeze bug

A small script generated invoices with extreme content in all three templates:

Input	Before	After
Terms: 20,000 characters	FAILED	OK - flows over 4 pages
Notes: 8,000 characters	FAILED	OK
Item description: 4,000 / 12,000 chars	FAILED	OK - 3 / 6 pages
Address with 90 lines	won't-fit / hang risk	clear error, instantly
100 taxes	won't-fit error	clear error
Long words, emoji, empty strings, tabs	OK	OK

> **Why 'failed' really meant 'freezes the app'**
> The pdf library keeps adding pages when a block that cannot be split is taller than a page. The only stop is `assert(sameCount++ <= maxPages)` - and asserts are **removed in release builds**. A customer pasting long terms would have frozen the app forever. Debug-mode tests could never show this; reading the library's code did.

Fixes:

- Notes, terms and instructions are now top-level text with `TextOverflow.span`, which always flows onto the next page.
- Long descriptions are cut into ~350-character pieces placed in **continuation rows**; padding and grid lines are adjusted so they still look like one item.
- Fields that must stay in one row (item name, unit, SKU, item taxes) have validator limits with helpful messages.
- Fixed blocks use `pw.Inseparable`: if they are too tall they fail *immediately* with 'taller than one page', instead of looping.

> **A fix that would have broken old users**
> The first version of the friendly error caught `PdfException`. Checking pdf 3.12 showed that class does not exist there (added in 3.13), and our pubspec allows 3.12. We match the error message instead, which both versions share.

2. Reading the code again

Found	Fix
Due date 'today' rejected when issueDate = DateTime.now() (midnight is before now)	compare calendar days

Found	Fix
Tax 12.345% printed as 12.35% while the math used 12.345%	show up to 4 decimals
Currency decimals only checked by assert (gone in release)	validator checks it
baseFontSize 0 or a huge margin gave a cryptic library error	'Invalid InvoiceTheme: ...'
README said 'offline', but the web preview loads pdf.js from unpkg.com	documented, with the self-hosting option
Network logos are downloaded again on every preview refresh	documented; use InvoiceLogo.memory
Example app used a Flutter 3.35 API but claimed older support	example requires 3.35 (package still 3.27+)

3. Every check, one last time

```
flutter analyze (package + example)        No issues
flutter test                               +248 passed
flutter test --platform chrome (Brave)     +129 passed
example tests                              +7 passed
integration test on macOS (real plugins)   +2 passed
oldest dependency versions                 +248 passed
dart doc                                   0 warnings, 0 errors
dart format                                clean
dart pub publish --dry-run                 only the known Phase 10 items
```

> **In simple words**
>
> Tests prove what you thought of. An audit looks for what you did not think of: extreme inputs, release-mode behaviour, old versions, and every sentence in the README.